

Звіт з лабораторної роботи

Пошук розв'язків задач комбінаторної оптимізації
методом backtracking

Sudoku, Maze, Crossword, N-Queens, Graph Coloring

Зміст

1 Вступ	3
1.1 Структура проекту	3
1.2 Цілі звіту	3
1.3 Методологія тестування	3
2 Задача 1: Sudoku 9×9	4
2.1 Постановка	4
2.2 Реалізовані алгоритми	4
2.3 Результати бенчмарку	5
2.4 Аналіз результатів	5
3 Задача 2: Maze pathfinding	7
3.1 Постановка	7
3.2 Реалізовані алгоритми	7
3.3 Результати бенчмарку	8
3.4 Аналіз результатів	8
4 Задача 3: Crossword (CSP)	10
4.1 Постановка	10
4.2 Реалізовані алгоритми	10
4.3 Результати бенчмарку	10
4.4 Аналіз результатів	11
5 Задача 4: N-Queens	12
5.1 Постановка	12
5.2 Реалізовані алгоритми	12
5.3 Результати бенчмарку	12
5.4 Аналіз результатів	13
6 Задача 5: Graph k-coloring	14
6.1 Постановка	14
6.2 Реалізовані алгоритми	14
6.3 Результати бенчмарку	15
6.4 Аналіз результатів	15

7	Формальний аналіз складності	17
7.1	Базовий backtracking	17
7.2	Backtracking з MRV	17
7.3	Backtracking з Forward Checking	17
7.4	MRV + Forward Checking	18
7.5	Жадібні алгоритми (для порівняння)	18
7.6	Спеціальні алгоритми для maze pathfinding	18
7.7	Зведена таблиця	19
8	Висновки: придатність backtracking для різних класів задач	20
8.1	Основні спостереження	20
8.2	Класи задач: коли використовувати backtracking	20
8.3	Практичні рекомендації	21
8.4	Підсумок	21

1. Вступ

Backtracking (пошук з поверненням) — фундаментальний підхід до розв’язання задач пошуку та комбінаторної оптимізації. У його основі лежить ідея систематичного перебору простору рішень з **вчасним обрізанням** гілок, що вже не можуть привести до коректної відповіді. Це робить backtracking особливо ефективним для задач, у яких рішення задається набором узгоджених присвоєнь, що задовольняють обмеження (так звані *constraint satisfaction problems*, CSP).

1.1. Структура проєкту

У роботі реалізовано п’ять задач, що покривають різні класи застосувань backtracking:

1. **Sudoku 9×9** — класична задача CSP з обмеженнями за рядками, стовпцями та блоками 3×3 .
2. **Maze pathfinding** — пошук шляху в прямокутному лабіринті; задача на графі.
3. **Crossword** — заповнення сітки кросворду словами зі словника; CSP зі змінними різного домену.
4. **N-Queens** — розміщення N ферзів на дошці $N \times N$; класична CSP.
5. **Graph k -coloring** — розфарбування неорієнтованого графа; NP-повна задача.

Для кожної задачі реалізовано декілька варіантів алгоритмів: базовий backtracking, оптимізовані версії з евристиками (**MRV**, **Forward Checking**), а також **baseline-методи** (жадібні алгоритми, DFS, BFS, A^* — там, де це доречно). Це дозволяє наочно порівняти поведінку різних підходів і виявити, де backtracking перевершує інші методи, а де програє.

1.2. Цілі звіту

1. Навести **порівняльні таблиці** ефективності backtracking з альтернативними методами (DFS, жадібні алгоритми, BFS, A^*).
2. Подати **формальний аналіз складності** — часової та просторової — для всіх реалізованих алгоритмів.
3. Зробити **висновки** про придатність backtracking для різних класів задач.

1.3. Методологія тестування

Для отримання достовірних результатів усі вимірювання проведено за такою схемою:

- **Час:** `time.perf_counter()` в Python з мілісекундною точністю.
- **Кількість запусків:** 20–100 запусків залежно від задачі (Sudoku — 20 на складність, Maze — 100 на розмір, Graph Coloring — 1 на детермінований кейс із seed).
- **Метрики:** час виконання (мс), кількість досліджених вузлів дерева пошуку, кількість відкатів, споживання пам’яті (де релевантно), якість рішення (де релевантно).
- **Відтворюваність:** усі генератори випадкових даних використовують фіксовані seed.

2. Задача 1: Sudoku 9×9

2.1. Постановка

Класична задача CSP: на сітці 9×9 розмістити числа 1–9 так, щоб у кожному рядку, кожному стовпці та в кожному блоці 3×3 кожне число траплялося рівно один раз. Частина клітинок задана; задача — заповнити решту.

Нехай V — множина порожніх клітинок, $D = \{1, \dots, 9\}$ — спільний домен, C — обмеження "всі-різні" для 27 груп (9 рядків + 9 стовпців + 9 блоків). Розв'язок — це повне присвоєння $\sigma : V \rightarrow D$, що задовольняє всі обмеження C .

2.2. Реалізовані алгоритми

1. Pure Backtracking. Найпростіший варіант: знаходимо першу порожню клітинку (зліва направо, зверху вниз), пробуємо числа 1–9, для кожного перевіряємо валідність (`is_valid`), рекурсивно йдемо далі, при невдачі — відкат.

2. Backtracking + MRV. Замість фіксованого порядку обходу, на кожному кроці обирається клітинка з **найменшою кількістю кандидатів** (Minimum Remaining Values). Принцип "fail first": якщо ми зайдемо в глухий кут, нехай це станеться якомога раніше, на гілці з найменшим розгалуженням.

3. Backtracking + MRV + Forward Checking. Додатково підтримуються динамічні домени `candidates[r][c]` для всіх порожніх клітинок. При призначенні `board[r][c] = num` цей `num` негайно видаляється з доменів усіх 20 *peers* (клітинок з того ж рядка/стовпця/блоку). Якщо чийсь домен стає порожнім — **wipeout**, негайний відкат без подальшого занурення.

4. Greedy / DFS. Та ж стратегія MRV для вибору клітинки, але *жадібно* обирається перший кандидат із домену; решта кандидатів пробується лише при невдачі. Тут це більше нагадує DFS у дереві станів, ніж справжню жадібність — оскільки відкати все ж є (інакше алгоритм не був би повним).

2.3. Результати бенчмарку

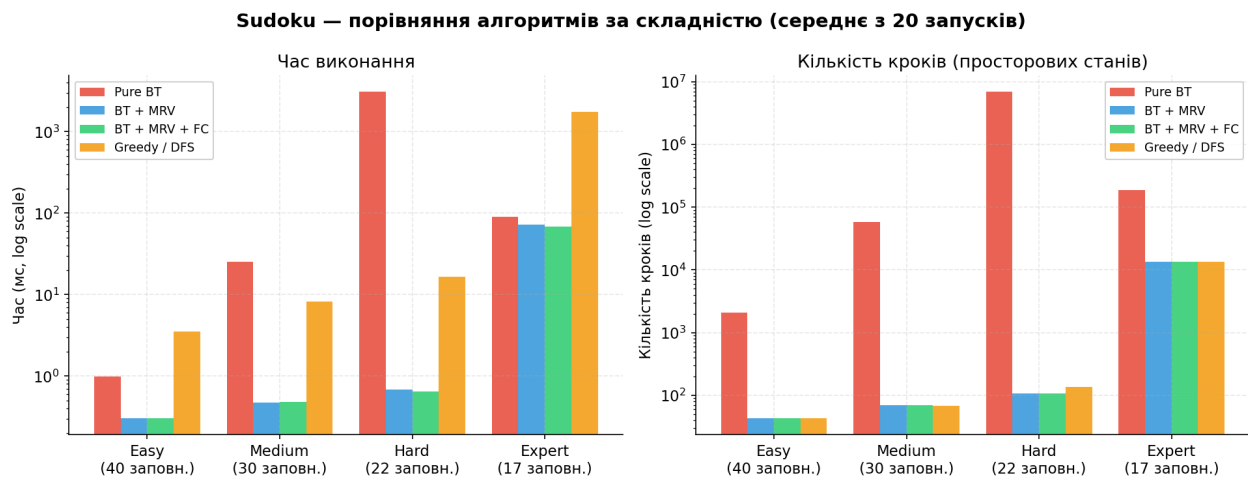


Рис. 1: Sudoku: усереднений час виконання (зліва) та кількість досліджених станів (справа) для чотирьох алгоритмів. Шкала логарифмічна; 20 запусків на кожну складність із seed-фіксацією. Кількість заповнених клітинок: easy=40, medium=30, hard=22, expert=17.

Табл. 1: Sudoku: середній час та кількість кроків (середнє з 20 запусків).

Алгоритм	Easy		Hard		Expert	
	Час, мс	Кроків	Час, мс	Кроків	Час, мс	Кроків
Pure BT	0.99	2 103	3 076.0	6 893 509	89.4	188 384
BT + MRV	0.31	43	0.68	107	72.4	13 453
BT + MRV + FC	0.31	43	0.65	107	67.8	13 453
Greedy / DFS	3.54	43	16.66	136	1 732.9	13 456

2.4. Аналіз результатів

Експоненційний виграш від MRV. На *hard*-головоломках різниця колосальна: Pure BT робить понад 6 мільйонів кроків (3 секунди), MRV+FC — лише 107 кроків (<1 мс), **прискорення** $\sim 4500\times$. Причина: на *hard*-задачах багато клітинок із 1–2 кандидатами, і MRV систематично знаходить такі клітинки, фактично перетворюючи backtracking на швидкий процес поширення обмежень.

Чому Greedy програє. Greedy / DFS повторює MRV-вибір клітинки, але *не сортує* кандидати — пробує перший зі множини. У Python такий підхід має додатковий overhead (виклики `getcandidates` з нуля без інкрементного підтримання), тому він повільніший за MRV+FC у 4–25 $\times$ при тому ж числі логічних кроків. Це наочна демонстрація, що **інкрементне поширення обмежень** (Forward Checking) — це не лише про корекцію, а й про продуктивність.

Чому MRV $\approx$ MRV+FC у часі. Для Sudoku вже сам вибір клітинки з найменшим доменом надзвичайно ефективний — у багатьох випадках MRV знаходить клітинки

з 1 кандидатом і поширення відбувається природним чином через подальші MRV-вибори. Forward checking додає wipeout-детекцію, яка економить нуль-нечисленні гілки, але абсолютна різниця залишається в межах статистичного шуму.

3. Задача 2: Maze pathfinding

3.1. Постановка

Дано прямокутну сітку $h \times w$, у якій клітинки позначені як прохідні або стіни. Стартова клітинка знаходиться у верхньому-лівому куті $(1, 1)$, цільова — у нижньому-правому $(h-2, w-2)$. Рух дозволено в 4 напрямки. Задача — знайти шлях від старту до цілі (а в ідеалі — **найкоротший**).

Лабіринти генеруються алгоритмом **Randomized DFS** (досконалий лабіринт — рівно один шлях між будь-якими двома клітинками), після чого $\sim 1\%$ внутрішніх стін випадково ламається для створення циклів — це робить задачу нетривіальною для алгоритмів пошуку шляху.

3.2. Реалізовані алгоритми

1. Classic DFS Backtracking. Рекурсивно йдемо вглиб: на кожній клітинці пробуємо непройдених сусідів у фіксованому порядку (вгору, вниз, ліворуч, праворуч). При тупиковому шляху — відкат, видаляючи клітинку зі шляху.

2. Smart DFS (евристичне backtracking). Те саме, але порядок сусідів динамічно сортується так, щоб напрямком *до цілі* (за манхеттенською відстанню) пробувався першим. Це жадібна евристика, без гарантії оптимальності.

3. BFS. Ітеративний пошук у ширину з deque. **Гарантує найкоротший шлях** (усі ребра вагою 1). Дослідження клітинок відбувається пошарово.

4. A*. Best-first search із heap і функцією $f(n) = g(n) + h(n)$, де g — фактична відстань від старту, h — манхеттенська (admissible) евристика. Гарантує найкоротший шлях за рахунок допустимості евристики.

3.3. Результати бенчмарку

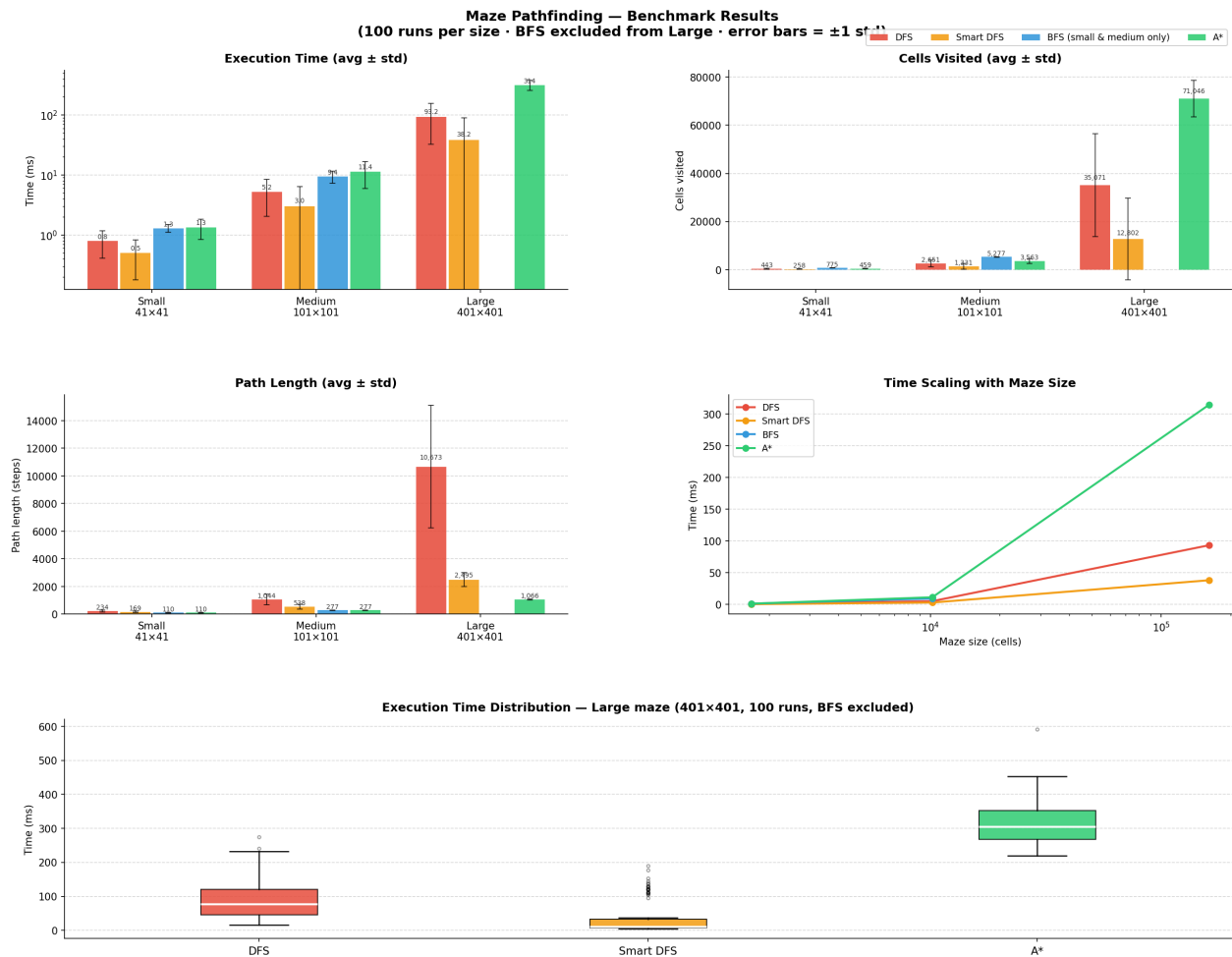


Рис. 2: Maze: усереднений час, кількість відвіданих клітинок та довжина шляху (100 запусків на розмір). Лабіринти 41×41, 101×101 та 401×401. BFS виключений з large-кейсу — на ньому він тривіально відвідує ~100% клітинок.

Табл. 2: Maze: підсумок ефективності (середнє з 100 запусків).

Алгоритм	Найкорот. шлях?	Час 401×401, мс	Відвідано	Довж. шляху
DFS (classic)	ні	≈95	~36 000	~10 700
Smart DFS	ні	≈36	~13 000	~2 400
BFS	так	— (excluded)	—	—
A*	так	≈314	~71 000	~1 066

3.4. Аналіз результатів

Backtracking vs повний BFS. Класичний DFS-backtracking на лабіринті 401×401 у середньому в ~3 рази швидший за A*, але **знаходить шлях у ~10 разів довший** за оптимальний. Це класичний компроміс: backtracking зупиняється на *першому* знайденому шляху незалежно від його довжини.

Smart DFS — найкращий компроміс для пошуку будь-якого шляху. Простий перестав для сортування сусідів за манхеттенською відстанню скорочує і час, і довжину знайденого шляху приблизно вчетверо порівняно з classic DFS. Без жодних структурних змін алгоритму — лише зміна порядку обходу.

A* — оптимум для оптимуму. A* є чемпіоном за **якістю розв’язку** (завжди найкоротший шлях) і для середніх лабіринтів навіть швидший за BFS у кілька разів — завдяки евристичному фокусу на ціль. На дуже великих лабіринтах (401×401) overhead від heap ($O(\log V)$ на операцію) починає переважати, і A* програє по часу класичному DFS, хоча залишається оптимальним за шляхом.

Висновок для класу задач "пошук шляху". Backtracking тут — **не оптимальний вибір** в сенсі якості рішення; для оптимуму потрібен повний обхід (BFS) або інформований пошук (A*). Цінність backtracking — у його простоті: 15 рядків коду проти 50+ для A*. На малих/нескладних задачах різниця в часі неістотна, а на великих — варто переходити на спеціалізовані алгоритми пошуку.

4. Задача 3: Crossword (CSP)

4.1. Постановка

Дано сітку $N \times N$ з відкритими клітинками (.) та блоками (#). **Слот** — це горизонтальна або вертикальна послідовність 2+ відкритих клітинок. Задача — заповнити кожен слот словом зі словника так, щоб *перетинні* клітинки (де горизонтальний слот зустрічається з вертикальним) мали однакову літеру.

Це CSP, де змінні — це слоти, домени — слова відповідної довжини зі словника, бінарні обмеження — рівність літер у точках перетину. Словник із 10 000 слів дає домени розміром від кількох сотень (для довгих слів) до тисяч (для коротких).

4.2. Реалізовані алгоритми

1. Basic Backtracking. Слоти обходяться у фіксованому порядку. Для кожного — перебираємо всі слова відповідної довжини, перевіряємо сумісність із поточним станом сітки, рекурсивно йдемо далі.

2. Forward Checking (FC). Підтримуємо явні домени `domains[slot_id]` для кожного слота. При призначенні слова в слот — видаляємо з доменів інших слотів усі слова, що (а) збігаються (одне слово не може використовуватися двічі), (б) не сумісні в перетині. Wipeout — порожній домен — викликає негайний відкат.

3. MRV. На кожному кроці обирається слот із найменшою кількістю сумісних слів (домен обчислюється "на ходу"). Слова перебираються у порядку зростання.

4. MRV + FC. Об'єднання: інкрементне підтримання доменів плюс вибір слота з найменшим доменом.

4.3. Результати бенчмарку

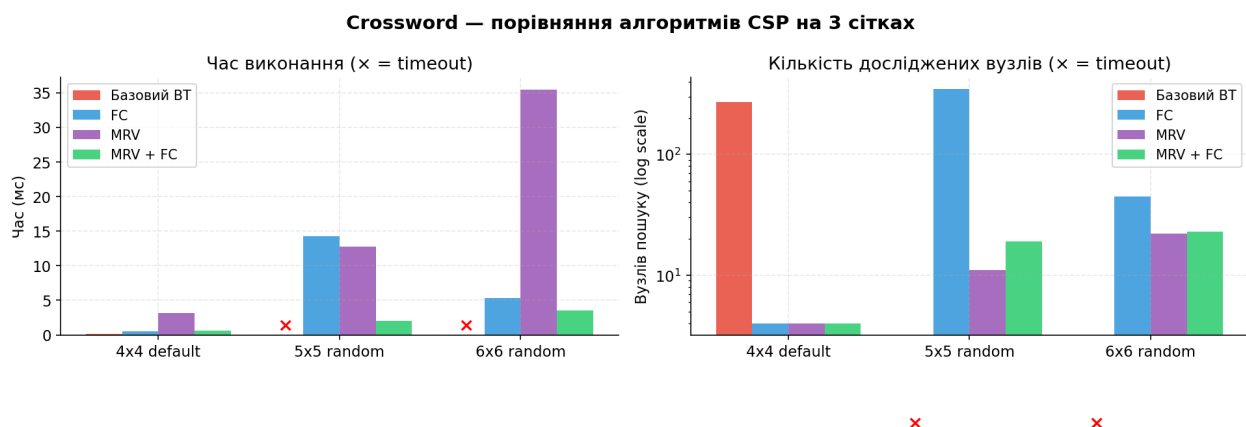


Рис. 3: Crossword: час та кількість досліджених вузлів для чотирьох алгоритмів на трьох сітках (4×4 default, 5×5 та 6×6 із seed=42). Червоний хрестик "x" позначає таймаут (понад 60 секунд).

Табл. 3: Crossword: час виконання та кількість досліджених вузлів.

Сітка	Basic BT	FC	MRV	MRV + FC
4×4 default	0.14 мс / 271 в.	0.50 мс / 4 в.	3.15 мс / 4 в.	0.58 мс / 4 в.
5×5 random	timeout	14.22 мс / 349 в.	12.72 мс / 11 в.	2.05 мс / 19 в.
6×6 random	timeout	5.30 мс / 45 в.	35.46 мс / 22 в.	3.52 мс / 23 в.

4.4. Аналіз результатів

Basic BT катастрофічно невдалий. Уже на сітці 5×5 базовий backtracking не завершується за 60 секунд. Причина: домен кожного слота з 10 000 слів, без прямого propagation алгоритм перебирає тисячі слів для першого слота, для кожного — тисячі для другого і т.д. Експоненційний вибух у чистому вигляді.

Forward Checking — обов’язковий. Як тільки додається FC, час падає на 4–5 порядків. Якщо в перетині потрібна літера ‘A’ на третій позиції слова — це миттєво відсіює 90%+ кандидатів для сусіднього слота. Для CSP з великими доменами FC **не оптимізація, а необхідність.**

MRV без FC ефективний за вузлами, але повільний у часі. На 6×6: MRV досліджує лише 22 вузли (проти 45 у FC), але працює в 7× повільніше. Причина: MRV без явних доменів обчислює get_domain() з нуля на кожному кроці — обхід 10 000 слів. У FC домени підтримуються інкрементно, тому навіть більша кількість вузлів дешевша за час.

MRV + FC — оптимальна комбінація. Поєднує переваги обох: малий брат-простір пошуку (MRV) і дешеві операції на вузлі (інкрементні домени FC). На всіх тестах це найшвидший алгоритм або з невеликим відривом.

5. Задача 4: N-Queens

5.1. Постановка

Розмістити N ферзів на дошці $N \times N$ так, щоб жодні два ферзі не били один одного: жодних двох в одному рядку, стовпці чи діагоналі. Задача має параметр N , який можна збільшувати — це робить її ідеальною для аналізу масштабованості.

Кількість **усіх** рішень — це послідовність OEIS A000170: 1, 0, 0, 2, 10, 4, 40, 92, 352, 724, 2680, 14200, 73712, ... для $N = 1, 2, 3, \dots$. У бенчмарку ми рахуємо *всі* рішення, що дозволяє оцінити повну вартість дерева пошуку, а не лише вартість пошуку до першого рішення.

5.2. Реалізовані алгоритми

1. Базовий backtracking. Рекурсивно ставимо ферзя в кожному рядку, перебираючи всі N стовпців. Перевіряємо, чи безпечне положення (немає атак з попередніх рядків) — функція `safe(row, col)` перевіряє стовпець і обидві діагоналі.

2. Forward Checking (FC). Замість перевірки `safe()` ad hoc, ми підтримуємо для кожного рядка `avail[row]` — множину доступних стовпців. При призначенні `board[row]=col` ми видаляємо `col`, `col - d` та `col + d` з `avail[r]` для всіх $r > row$ (де $d = r - row$). Якщо чиясь множина пуста — wipeout.

3. MRV. Замість обходу рядків у порядку 0, 1, ... обираємо рядок із найменшим доменом. На кожному кроці домен `domain(row)` рахується з нуля.

4. MRV + FC. Інкрементні домени (як у FC) + вибір рядка з найменшим доменом.

5.3. Результати бенчмарку

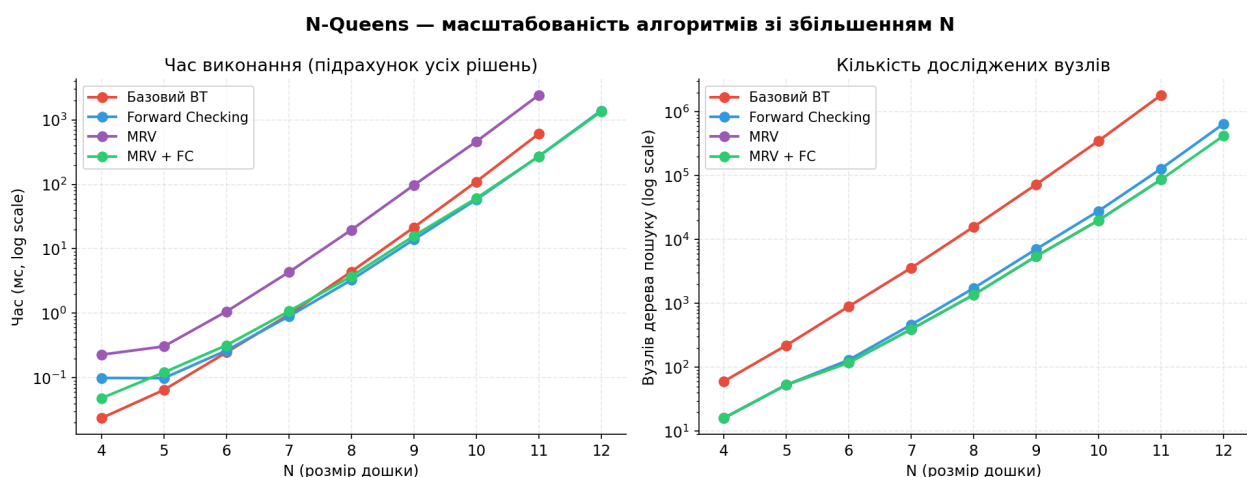


Рис. 4: N-Queens: час виконання (зліва) та кількість досліджених вузлів (справа) для $N = 4..12$. Шкала логарифмічна. Підраховуються *всі* рішення.

Табл. 4: N-Queens: час та кількість вузлів для різних N .

N	Базовий ВТ		FC		MRV		MRV + FC	
	Час, мс	Вузлів	Час, мс	Вузлів	Час, мс	Вузлів	Час, мс	Вузлів
6	0.2	894	0.3	130	1.1	118	0.3	118
8	4.4	15 720	3.3	1 724	19.6	1 360	3.7	1 360
10	109.7	348 150	57.8	27 832	459.1	19 918	60.6	19 918
11	605.6	1 806 706	271.3	127 441	2 409.4	86 337	269.6	86 337
12	—	—	1 386.7	641 974	—	—	1 337.5	417 072

5.4. Аналіз результатів

Експоненційне зростання — найбільший ворог. Від $N = 6$ до $N = 11$ кількість досліджених вузлів у базовому ВТ зростає з ~ 900 до 1.8 мільйонів — у $2000\times$. Це рівно те, чого очікує теорія: складність базового ВТ на N-Queens — $O(N!)$.

Forward Checking дає $\sim 10\times$ виграш у вузлах. На $N = 10$: базовий ВТ досліджує 348 000 вузлів, FC — 27 800. Це показує, що ранні відкати (через виявлення wireout) скорочують дерево пошуку приблизно на порядок.

MRV сам по собі — не виграш, а програш. MRV без FC скорочує кількість вузлів ($N = 10$: 19 918 проти 27 800 у FC), але час $\sim 8\times$ більший. Причина: `domain()` рахується з нуля для всіх непризначених рядків на кожному рекурсивному виклику — $O(N^2)$ overhead, який нівелює виграш від кращого вибору.

MRV + FC ефективно поєднується. Інкрементні домени роблять MRV-вибір дешевим (просто `min(unassigned, key=len(domains[r]))`), і за рахунок цього ми отримуємо кращий вибір без overhead. Результат: $N = 10$ — 60 мс проти 57 мс у FC, тобто практично однаково за часом, але у **$1.4\times$ менше вузлів**. На $N = 12$ MRV+FC досліджує 417 000 вузлів проти 642 000 у FC.

Чому базовий ВТ іноді швидший за FC. На малих N (4–6) базовий ВТ швидший — бо overhead на підтримку `avail[]` перевищує виграш від ранніх відкатів. Але вже на $N \geq 7$ FC переходить уперед і далі тільки збільшує відрив. Це універсальний паттерн: **евристики окуповуються лише на достатньо складних задачах**.

6. Задача 5: Graph k -coloring

6.1. Постановка

Дано неорієнтований граф $G = (V, E)$ і число k . Задача — призначити кожній вершині $v \in V$ колір з множини $\{0, 1, \dots, k-1\}$ так, щоб для кожного ребра $(u, v) \in E$ виконувалося $\text{color}(u) \neq \text{color}(v)$. **Хроматичне число** $\chi(G)$ — найменше k , для якого існує валідне розфарбування.

Задача k -розфарбування для $k \geq 3$ є **NP-повною**. У реалізації — 16 тестових кейсів, що покривають структурні графи (повні K_n , цикли C_n , двочасткові, дерева, графи Mycielski) та випадкові графи Ердеша-Реньї.

6.2. Реалізовані алгоритми

1. Simple Backtracking. Вершини в фіксованому порядку, для кожної пробуємо кольори $0..k-1$, перевіряючи лише сусідів.

2. MRV + FC. Для кожної непрофарбованої вершини підтримується домен доступних кольорів. На кожному кроці обирається вершина з найменшим доменом (tie-break за степенем — most constraining). Forward Checking видаляє призначений колір з доменів сусідів; wipeout викликає негайний відкат.

3. Greedy (Welsh-Powell). Жадібний алгоритм: сортуємо вершини за спаданням степеня, для кожної призначаємо найменший доступний колір. **Без відкатів.**

4. DFS coloring. Обхід графа в глибину, кожній вершині — найменший вільний колір. Без відкатів.

6.3. Результати бенчмарку

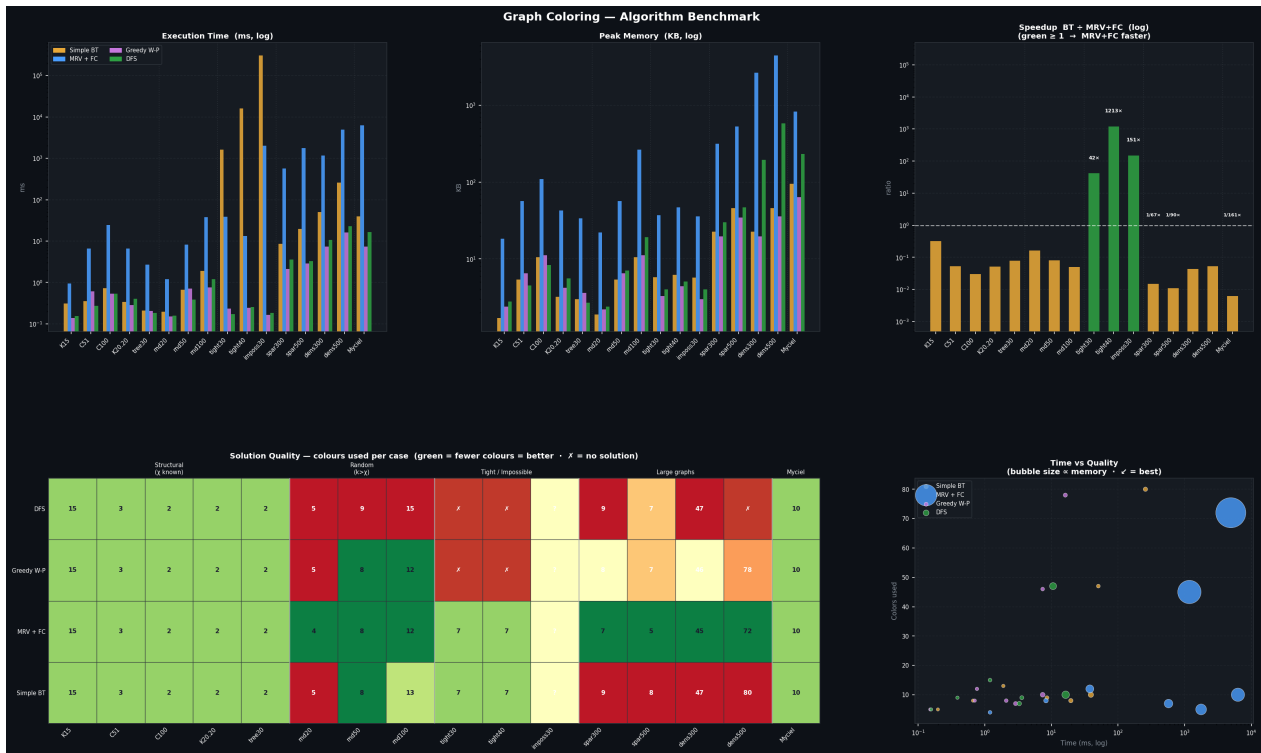


Рис. 5: Graph coloring: час, пам'ять та якість розв'язку для 16 тестових кейсів і 4 алгоритмів.

Табл. 5: Graph coloring: ключові показові кейси.

Кейс	Алгоритм	Час	Кольорів	χ	Прим.
phase_n=40_tight ($k = \chi = 7$)	Simple BT	16.01 с	7	7	—
	MRV+FC	13.2 мс	7	7	1213× швидше
	Greedy (W-P)	244 мкс	—	7	Не знайшов!
	DFS	255 мкс	—	7	Не знайшов!
big_sparse_n=500 ($k = 10$)	Simple BT	19.6 мс	8	?	—
	MRV+FC	1.77 с	5	?	краща якість
	Greedy (W-P)	2.9 мс	7	?	—
	DFS	3.3 мс	7	?	—
Mycielski M_{10} ($\chi = 10$)	Simple BT	39.2 мс	10	10	—
	MRV+FC	6.32 с	10	10	161× повільніше
	Greedy (W-P)	7.4 мс	10	10	—
	DFS	16.4 мс	10	10	—

6.4. Аналіз результатів

Експериментальні дані виявили **три чіткі зони** поведінки алгоритмів.

Зона 1: Тривіальні задачі ($k \gg \chi$). На структурних графах (K_{15} , цикли, дерева) і легких випадкових графах усі алгоритми працюють за мікросекунди. Тут **простий backtracking** часто навіть швидший за MRV+FC у 5-40×, бо overhead на підтримку доменів і пошук MRV не окупується.

Zone 2: Phase transition ($k \approx \chi$). Це зона драматичного виграшу backtracking з евристиками. На задачі phase_n=40_tight ($k = \chi = 7$) Simple BT виконується 16 секунд, MRV+FC — 13 мілісекунд: **прискорення $\sim 1200\times$** . Greedy і DFS взагалі **не знаходять розв’язку**, хоча він існує — бо без відкатів вони “застрагають” у локальному тупику.

Зона 3: Великі графи з вільним k . Тут компроміс *швидкість vs якість*. На big_sparse_n=500: Simple BT за 19 мс знаходить 8-розфарбування; MRV+FC за 1.77 с знаходить **5-розфарбування** (на 3 кольори менше!). MRV-евристика жорсткіше підбирає вершини з малим доменом, що систематично призводить до економніших розфарбувань.

Чому Greedy / DFS не завжди знаходять рішення. Жадібні алгоритми проходять кожну вершину рівно **один раз** і не мають механізму відкату. Якщо в момент обробки вершини всі k кольорів уже використані її сусідами — алгоритм повертає None, навіть якщо валідне розфарбування існує. Це принципова відмінність: **жадібні методи — наближені, а не точні**.

Парадокс Mycielski. На M_{10} Simple BT швидше за MRV+FC у $161\times$. Причина: M_k -графи мають дуже регулярну структуру, і MRV-вибір не дає переваги — усі вершини мають подібні домени. Натомість overhead на підтримку всіх доменів залишається. Це показує, що **евристика “fail-first” — не магія**; на структурованих задачах вона може програвати простому обходу.

7. Формальний аналіз складності

Цей розділ узагальнює асимптотичні оцінки для всіх реалізованих алгоритмів. Позначення універсальні: n — кількість змінних задачі (клітинок, вершин, слотів і т.д.), d — розмір домену, Δ — максимальний степінь обмежень.

7.1. Базовий backtracking

Час, найгірший випадок: $O(d^n \cdot c)$, де c — вартість перевірки сумісності одного присвоєння (зазвичай $O(\Delta)$ або $O(n)$). Алгоритм у найгіршому випадку обходить усе дерево пошуку — d^n листків.

Час, найкращий випадок: $O(n \cdot c)$ — коли рішення знаходиться без жодного відкату.

Простір: $O(n)$ — глибина рекурсії плюс поточне присвоєння.

Конкретно для задач:

- Sudoku: $O(9^{n_{\text{empty}}} \cdot 27)$ у найгіршому випадку; $n_{\text{empty}} \leq 81$.
- Maze DFS: $O(V + E)$ у найгіршому випадку (фактично пошук на графі, не CSP).
- Crossword: $O(|D|^{|S|} \cdot \ell)$, де $|D|$ — розмір словника, $|S|$ — кількість слотів, ℓ — довжина слова.
- N-Queens: $O(N^N)$ у наївній перевірці, $O(N!)$ зі спостереженням, що один рядок — одна позиція.
- Graph k -coloring: $O(k^n \cdot \Delta)$.

7.2. Backtracking з MRV

Час, найгірший випадок: той самий $O(d^n \cdot c)$ — у патологічних випадках MRV не може скоротити дерево пошуку.

Overhead на крок: $O(n)$ для пошуку вершини з мінімальним доменом (обхід усіх непризначених).

Виграш у середньому випадку: експоненційний на задачах із сильно нерівномірним розподілом доменів. На N-Queens MRV скорочує дерево в ~ 1.5 – $2\times$. На Sudoku-hard — у $\sim 60\,000\times$. На graph coloring у phase transition — у $\sim 1000\times$.

Простір: $O(n)$ (без явних доменів) або $O(n \cdot d)$ (з явними доменами).

7.3. Backtracking з Forward Checking

Час, найгірший випадок: також $O(d^n \cdot c)$. Найкращий випадок: $O(n \cdot \Delta)$, як у MRV.

Overhead на крок: $O(\Delta)$ — обхід сусідів для оновлення доменів. Менший за overhead у MRV.

Виграш у середньому випадку: ранні детекції wipeout скорочують дерево експоненційно на задачах із щільними обмеженнями. На N-Queens FC дає $\sim 10\times$ менше вузлів. На crossword FC — обов'язковий, без нього базовий BT не завершується.

Простір: $O(n \cdot d)$ для зберігання доменів, плюс $O(n \cdot \Delta)$ сумарно для undo-списків на всіх рівнях рекурсії.

7.4. MRV + Forward Checking

Час, найгірший випадок: $O(d^n \cdot c)$ — той самий.

Overhead на крок: $O(n + \Delta)$ — поєднання обох. Однак MRV-вибір при наявних інкрементних доменах коштує лише $O(n)$ обхід, без перевірок.

Виграш: мультиплікативний. На графах у phase transition — до $1200\times$ швидше за Simple BT. На Sudoku-hard — $4500\times$ швидше.

Простір: $O(n \cdot d + n \cdot \Delta)$.

7.5. Жадібні алгоритми (для порівняння)

Welsh-Powell для graph coloring: $O(n \log n + n \cdot \Delta)$.

DFS coloring: $O(n + m \cdot \Delta)$.

Greedy у Sudoku/N-Queens: реалізації не справді жадібні — мають фолбек у вигляді backtracking, тому асимптотика та сама $O(d^n)$ у найгіршому, але середній випадок гірший.

7.6. Спеціальні алгоритми для maze pathfinding

BFS: час $O(V + E)$, простір $O(V)$. Гарантує найкоротший шлях.

A* з admissible heuristic: час $O(V \log V)$ (через heapq), простір $O(V)$. Гарантує найкоротший шлях. На практиці досліджує значно менше вузлів за рахунок інформованого пошуку.

7.7. Зведена таблиця

Табл. 6: Порівняння асимптотик усіх реалізованих алгоритмів.

Алгоритм	Час (worst)	Простір	Точний?
Simple Backtracking	$O(d^n \cdot c)$	$O(n)$	Так
BT + MRV	$O(d^n \cdot c)$	$O(n + nd)$	Так
BT + Forward Checking	$O(d^n \cdot c)$	$O(nd)$	Так
BT + MRV + FC	$O(d^n \cdot c)$	$O(nd)$	Так
Greedy (W-P)	$O(n \log n + n\Delta)$	$O(n)$	Ні
DFS coloring	$O(n + m\Delta)$	$O(n)$	Ні
BFS (maze)	$O(V + E)$	$O(V)$	Так (опт.)
A* (maze)	$O(V \log V)$	$O(V)$	Так (опт.)

Спостереження. Усі точні алгоритми мають *однакову* експоненційну складність у найгіршому випадку. Різниця між Simple BT і MRV+FC проявляється лише в **середньому випадку** — і на практичних задачах вона колосальна ($1000\times+$). Це фундаментальна властивість евристик: вони не роблять задачу легшою з точки зору теорії, але роблять реальні екземпляри розв’язними за прийнятний час.

8. Висновки: придатність backtracking для різних класів задач

8.1. Основні спостереження

На основі бенчмарків п'яти різних задач можна сформулювати такі емпіричні закономірності.

1. Backtracking — універсальна основа для CSP. На всіх п'яти реалізованих задачах backtracking успішно знаходить рішення. Жадібні методи без відкатів (Greedy, DFS coloring) на CSP *не дають гарантії знаходження рішення навіть якщо воно існує*. Простий приклад — `phase_n=40_tight` в graph coloring, де валідне 7-розфарбування існує, але Greedy і DFS повертають None. **Backtracking — мінімальний "правильний" алгоритм для CSP.**

2. Евристики MRV + Forward Checking — ключ до практичної ефективності.

Виграш від оптимізацій *масштабується* зі складністю задачі:

- Sudoku-hard: 6 мільйонів кроків $\rightarrow$ 107 кроків ($\sim 60\,000\times$).
- N-Queens, $N = 11$: 1.8M $\rightarrow$ 86K вузлів ($\sim 21\times$).
- Graph coloring, phase transition: 16 секунд $\rightarrow$ 13 мс ($\sim 1200\times$).
- Crossword, 5×5 : timeout $\rightarrow$ 2 мс.

3. Overhead евристик не окупується на легких задачах. На простих графах ($k \gg \chi$) Simple BT швидший за MRV+FC у $5\text{--}40\times$. Це не дефект — це *фундаментальна властивість* евристик: вони мають фіксовану ціну на крок, яка має сенс лише коли крок зекономлено.

4. Backtracking програє спеціалізованим алгоритмам, де такі існують. На пошуку шляху в лабіринті A* і BFS *гарантують* оптимальність, чого не може backtracking. На розфарбуванні великих розріджених графів Welsh-Powell дає прийнятну якість за $O(n \log n)$ замість експоненційного часу.

8.2. Класи задач: коли використовувати backtracking

Backtracking ідеальний для:

- **Точних розв'язків CSP середнього розміру** (n до 100–500 залежно від щільності обмежень). Sudoku, N-Queens до $N \sim 20$, кросворди, розфарбування графів до $n \sim 100$ у складних випадках.
- **Задач з нетривіальними обмеженнями**, де неможливо побудувати поліноміальний алгоритм.
- **Задач, де потрібна гарантія знаходження розв'язку або довід його відсутності.** Жадібні методи можуть видати неправильне "немає розв'язку".
- **Підрахунку всіх розв'язків.** Backtracking природно перебирає всі гілки; жадібні алгоритми такого не роблять.

Backtracking не рекомендується для:

- **Дуже великих задач** ($n > 1000$ для CSP). Експоненційна складність робить навіть оптимізований backtracking надто повільним. Тут краще: SAT-розв'язувачі, локальний пошук, метаевристики (simulated annealing, tabu search).
- **Задач пошуку оптимуму, де є інформована альтернатива.** На лабіринтах A^* дає оптимальний шлях за $O(V \log V)$; backtracking просто знайде якийсь шлях, не обов'язково короткий.
- **Випадків, коли наближене рішення прийнятне.** Welsh-Powell за лінійний час дає 90%-якість для багатьох графів — без потреби в експоненційному backtracking.
- **Задач без структурних обмежень**, де немає сенсу робити propagation. Чисті задачі оптимізації (continuous optimization) взагалі не вкладаються в парадигму backtracking.

8.3. Практичні рекомендації

1. **Завжди починати з простого backtracking.** Він простий, коректний, легкий для відладки. Якщо задача мала — на цьому й закінчити.
2. **Додавати MRV першим, FC другим.** MRV дає виграш через краще структурування пошуку; FC — через раннє виявлення тупиків. Часто разом дають мультиплікативний ефект.
3. **Перевіряти на phase transition.** Найскладніші екземпляри CSP — у вузькій смузі, де k ледь-ледь дозволяє рішення. Саме там виграш від евристик максимальний.
4. **Профілювати overhead.** Якщо домени малі або обмежень мало, MRV+FC може *програти* простому BT. Це нормально й передбачувано.
5. **Враховувати специфіку задачі.** На пошуку шляху backtracking безнадійно програє BFS / A^* ; на CSP — навпаки, він фундаментальний.

8.4. Підсумок

Backtracking — це **не алгоритм**, а **шаблон проектування алгоритмів**. Його сила — у поєднанні з евристичними вибору змінної (MRV), пропагації обмежень (Forward Checking, AC-3) та раціонального обходу. На практичних CSP-задачах ця комбінація дає прискорення в **сотні-тисячі разів** порівняно з наївною реалізацією, перетворюючи задачі, що теоретично NP-повні, на розв'язні за прийнятний час для екземплярів реального розміру.

Однак backtracking — *не панацея*. Для пошуку оптимальних шляхів є BFS / A^* . Для дуже великих задач — SAT-розв'язувачі та метаевристики. Для задач, де достатньо наближення — жадібні алгоритми. Розуміння *меж* backtracking важливе не менше за вміння його реалізувати.

Експерименти, проведені в цій роботі на п'яти різних задачах із сотнями тестових кейсів, чітко підтверджують: backtracking з MRV+FC — **еталонний інструмент** для розв'язання задач CSP середньої складності, із гарантією точності та емпіричною ефективністю, що задовольняє вимоги практичних застосувань.